PATENT APPLICATION DISCLOSURE DRAFT

Budgeted Contract Repair for Agent Workflows

Working draft for review only. This is not legal advice or a filing-ready specification and makes no assertion of novelty or patentability. Personal, applicant, and filing details not supplied are left blank. Confirm all technical statements against the implementation before filing.

1. Full name, nationality and address of applicant(s)

Full Name: _____

Nationality: _____

Address: _____

_____

2. Full name, nationality, address, email and phone number of inventor(s)

Full name (including middle name)	Nationality	Address / institution	Email	Phone	Gender

3. Title of the invention

Budgeted Contract Repair for Agent Workflows Using Constraint-Ranked Repair Subgraphs and Verifiable Execution Certificates.

4. Technical field of the invention

• Computer-implemented orchestration of software-agent and tool workflows represented as directed graphs.

• Runtime validation of typed data artifacts against machine-evaluable interface and policy contracts.

• Budget-, latency-, privacy-, and risk-aware selection and execution of workflow repairs.

• Cryptographic integrity records for workflow execution and repair decisions.

5. Prior art

To be completed after a documented patent and non-patent literature search. No specific references or novelty conclusions are asserted here. Search areas: visual workflow orchestration; multi-agent workflow recovery; data/interface contract validation; automated plan or subgraph repair; budget-aware scheduling; and cryptographic provenance or Merkle logging.

Databases, search terms, date, closest references, and differences: _____

6. Objects of the invention

• Detect failed contracts on artifacts flowing between workflow nodes and identify violated constraints.

• Select a feasible repair subgraph from candidates by minimizing a weighted objective involving execution cost, latency, privacy risk, and residual failure risk.

• Reject candidates exceeding remaining budget or violating graph, privacy, or interface constraints.

• Insert a selected repair into a live execution plan while preserving acyclicity, then revalidate its output.

• Create a hash-based certificate identifying the artifact, contract, failure, candidate decision, and repair result.

• Support later integrity checks and reproducible inspection of the repair decision.

7. Synopsis

MindWeave-R is a proposed computer-implemented system for recovering agent workflows when an edge contract fails. A workflow is a directed graph whose nodes transform typed artifacts and whose edges specify requirements such as schema fields, provenance, confidence, privacy classification, freshness, and maximum cost. On failure, the system creates a structured failure report and retrieves candidate repair subgraphs. It filters candidates against hard constraints, including remaining budget, privacy policy, and graph validity, then ranks feasible candidates using configurable weights for cost, latency, privacy risk, and residual failure risk. The selected subgraph is inserted into the active execution plan and its output is checked against the failed contract. The system records cryptographic hashes of the relevant evidence in a repair certificate, optionally organized as a Merkle tree. This is a proposed mechanism; implementation coverage and performance require verification.

8. Brief description of drawings

Figure 1: Contract failure detection, candidate filtering and ranking, repair execution, revalidation, and certificate generation.

Figure 2: Constrained candidate-selection flow.

Figure 3: Hash structure of a repair certificate. Formal drawings should be prepared and reconciled with the final specification.

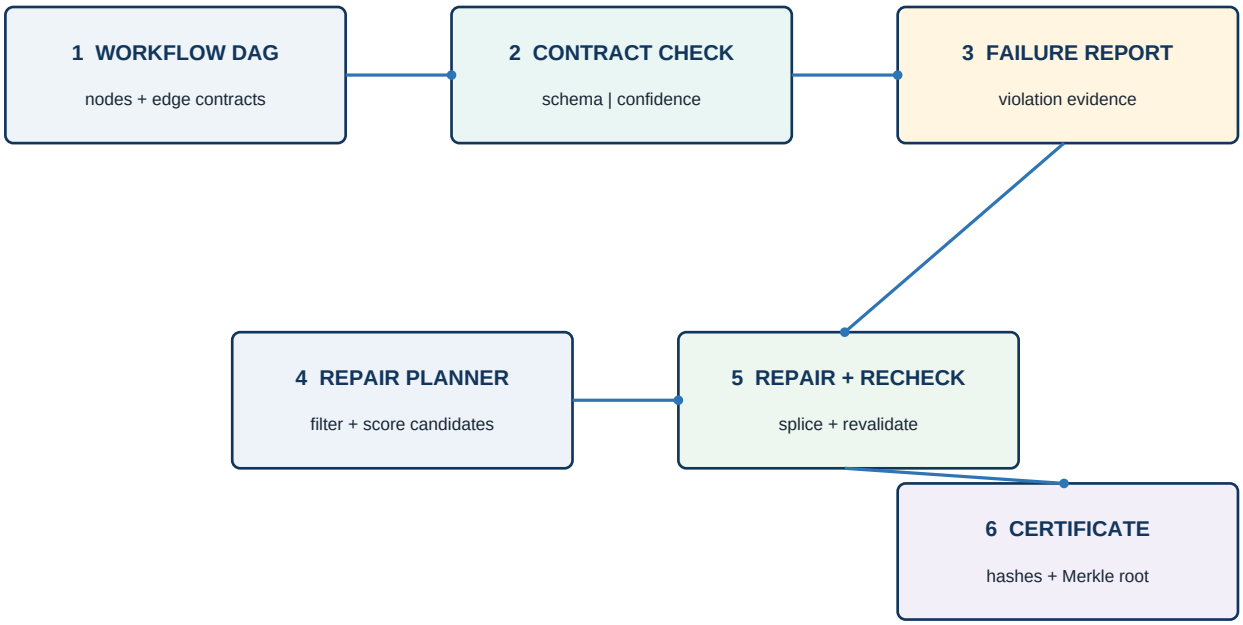

FIGURE 1 Runtime repair architecture - artifact and decision paths are recorded for audit.

Figure 1. Runtime repair architecture for a failed workflow edge.

9. Detailed description of the invention

Definitions. A workflow graph G=(V,E) contains agent, tool, or module nodes. An edge carries an artifact and a machine-readable contract. An artifact is structured output with metadata such as schema, provenance, confidence, privacy level, creation time, and execution cost. A repair subgraph is a candidate sequence or DAG of operations intended to address one or more contract violations.

Contract evaluation. Before downstream consumption, an evaluator checks the artifact against edge requirements. A contract may specify required fields or types, provenance, minimum confidence, allowed privacy level, maximum age, and cost ceiling. The evaluator returns a pass or a failure report with violation codes and observed values.

Candidate feasibility. For a failure, the planner retrieves repair subgraphs associated with the failure codes. Candidate metadata can include estimated cost, latency, privacy risk, residual failure risk, and expected fixes. Candidates are excluded when they exceed remaining budget, violate privacy rules, create a cycle, lack required inputs, or breach configured hard constraints.

Ranking. In one embodiment, J(r)=alpha*C(r)+beta*L(r)+gamma*P(r)+delta*R(r), where C is estimated cost, L latency, P privacy risk, R residual failure risk, and the weights are configurable and non-negative. The lowest-score feasible candidate is selected. If none is feasible, execution halts or follows a configured escalation route; the system does not silently treat the failed contract as satisfied.

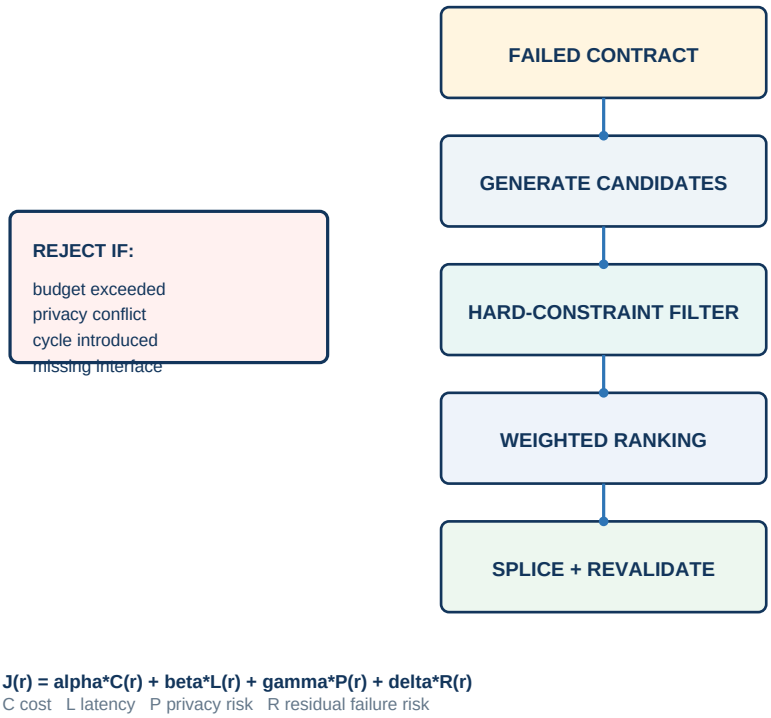

Figure 2. Candidate filtering, weighted ranking, repair execution, and contract revalidation.

9. Detailed description of the invention continued

Graph splice and revalidation. The selected repair subgraph is inserted between the producer and consumer of the failed edge. The system verifies interface compatibility and acyclicity, executes within the remaining budget, and evaluates the result against the original contract. Downstream execution resumes only on success. Otherwise a bounded alternative may be tried or execution terminates with a structured failure.

Repair certificate. The system hashes canonical representations of the original artifact, contract, failure report, candidate set, selected repair, repaired output, and trace events. Event hashes may be combined into a Merkle tree; a stored root and inclusion proofs can support integrity verification. Hashes establish integrity of recorded data, not truth of model output or completeness of inputs.

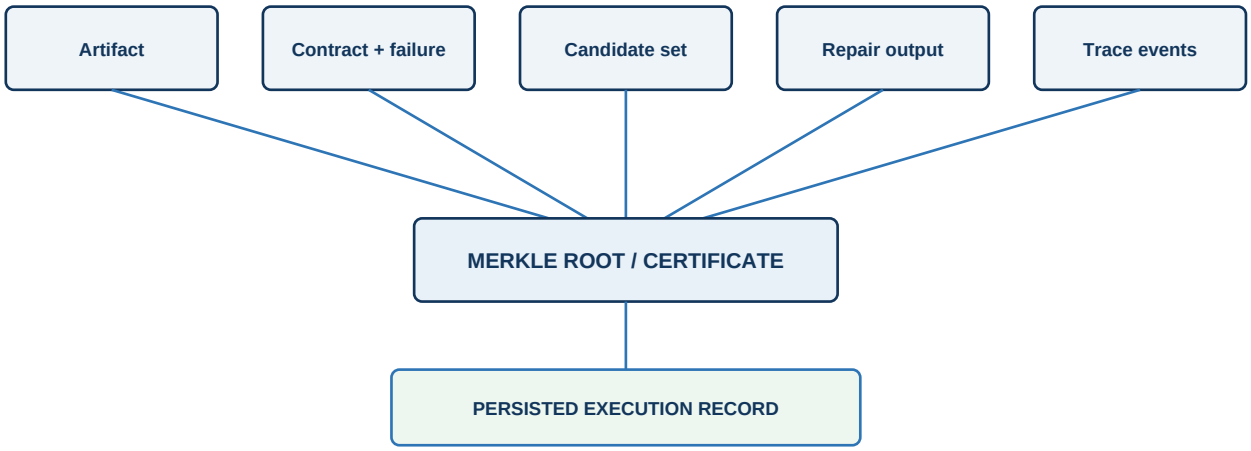

FIGURE 3 Canonical evidence hashes support integrity checks of the recorded repair trace.

Figure 3. Verifiable repair-certificate evidence and Merkle-root record.

Example implementation. A local embodiment may use typed Python services, NetworkX for graph validation and topological execution, SQLite for workflow and trace records, and a visual web canvas. Deterministic demo mode may emulate agent outputs; mock artifacts should be distinguished from real model/tool outputs. These are examples, not limitations.

10. Best method of performance of the invention

1. Receive a workflow DAG, node interfaces, edge contracts, candidate repair library, objective weights, and execution budget.

2. Validate the graph, then execute nodes in topological order while routing upstream artifacts to downstream inputs.

3. Evaluate each outgoing artifact against schema, provenance, confidence, privacy, freshness, and cost constraints.

4. On failure, record violation codes and retrieve candidate repair subgraphs associated with those codes.

5. Filter candidates against budget, privacy, interface compatibility, acyclicity, and retry limits.

6. Score feasible candidates with the configured objective and select the lowest-scoring candidate.

7. Insert and execute the selected repair; recheck its output against the original contract.

8. Resume downstream execution only after a pass; otherwise try a bounded alternative or safely halt.

9. Hash the inputs, failure, candidate decision, repair output, and trace; persist the certificate and optional Merkle root.

11. Claims

1. A computer-implemented method for recovering workflow execution, comprising executing nodes of a directed workflow graph; evaluating an artifact at an edge against a machine-readable contract; generating a failure report identifying contract violations; obtaining candidate repair subgraphs; filtering candidates according to at least remaining execution budget and graph validity; ranking filtered candidates using a score comprising weighted cost, latency, privacy risk, and residual failure risk; selecting a candidate; inserting and executing the selected repair subgraph; re-evaluating a resulting artifact against the contract; and recording a cryptographic certificate derived from data identifying the artifact, contract, failure report, selected candidate, and resulting artifact.

2. The method of claim 1, wherein the contract specifies one or more of required schema fields, provenance, minimum confidence, allowed privacy classification, maximum freshness age, or maximum execution cost.

3. The method of claim 1, wherein filtering excludes a candidate whose estimated cost exceeds a remaining workflow budget.

4. The method of claim 1, wherein graph validity includes verification that insertion does not create a directed cycle.

5. The method of claim 1, wherein the score is J(r)=alpha*C(r)+beta*L(r)+gamma*P(r)+delta*R(r), using configurable non-negative weights.

6. The method of claim 1, wherein downstream execution resumes only after contract revalidation succeeds, and otherwise the system tries a bounded alternative or halts.

7. The method of claim 1, wherein the certificate comprises event hashes combined into a Merkle tree and a stored root enabling integrity verification of at least a portion of an execution trace.

8. A workflow orchestration system comprising processors and memory storing instructions that cause performance of the method of any one of claims 1 to 7.

9. A non-transitory computer-readable medium storing instructions that cause performance of the method of any one of claims 1 to 7.

12. Inventive step (proposed contribution; subject to prior-art review)

The proposed technical contribution is the combination of contract-specific failure diagnosis, feasibility filtering of repair subgraphs under budget and policy constraints, weighted selection using cost/latency/privacy/residual-risk factors, runtime graph insertion followed by contract revalidation, and a cryptographically verifiable record of the repair decision and outcome. Whether this is new, inventive, or patent-eligible requires a prior-art search and qualified legal review. No conclusion of novelty or patentability is made.

13. Industrial application

• Agent-workflow orchestration in local or enterprise deployments.

• Research and evidence-processing pipelines requiring provenance.

• Document and content review workflows subject to privacy or policy constraints.

• Audited workflows where failed intermediate outputs must not silently pass downstream.

• Cloud, on-premises, or edge systems requiring bounded recovery under compute budgets.

14. Abstract

A computer-implemented method and system recover an agent workflow when an artifact fails a machine-readable edge contract. The system detects violations involving schema, provenance, confidence, privacy, freshness, or execution budget and obtains candidate repair subgraphs. It excludes candidates that violate constraints such as remaining budget, privacy policy, or graph acyclicity, and ranks feasible candidates using a configurable objective comprising cost, latency, privacy risk, and residual failure risk. A selected repair subgraph is inserted and executed. Downstream execution resumes only after the repaired artifact passes contract revalidation; otherwise a bounded alternative is attempted or execution halts. Cryptographic hashes of the artifact, contract, failure, candidate decision, output, and trace may be combined in a Merkle structure to form an integrity-verifiable repair certificate.

15. Drawings

Figures 1 to 3 are included within the specification above. Before official filing, redraw the figures according to the applicable patent-office drawing rules and verify that every label and referenced feature is consistent with the final claims.

Pre-filing review item	Status
Applicant and inventor particulars completed	Pending
Prior-art search recorded	Pending
Technical description checked against implementation	Pending
Claims reviewed by a patent professional	Pending
Drawing rules checked for filing jurisdiction	Pending

Applicant/inventor confirmation and filing particulars: _____

_____